

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-ab3024aecb55cda3a.jsonl`  
- SHA-256: `05a10023cb4bf101fd703bc8b8121ed4a5909eb577944ebc545719044b6e1602`  
- Source modified: `2026-07-02T03:04:06+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T03:02:02.139Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.

A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/khelsutra (avidullu/khelsutra) ? SPA web/ + marketing site/:

```
{  
  "title": "All 15 engine line-number contract anchors in the SPA API client are stale, and will drift further when engine PR #390 (main.py split) merges",  
  "category": "doc-rot",  
  "severity": "P2",  
  "file": "khelsutra/web/src/api/client.ts",  
  "line": 87,  
  "evidence": "web/src/api/client.ts:87,105,110,156,223,228,238,247,256,288,297 and web/src/api/types.ts:2-3,33,143,172 cite '[main.py:260]' (process), ':303' (jobs), ':447' (videos), ':331' (query), ':307' (highlights), ':121' (config), '[uploads.py:64/89/123/147]'. Actual engine routes today: /api/process main.py:1355, /api/jobs main.py:1457, /api/videos main.py:521, /api/query main.py:1523, /api/highlights main.py:1492, /api/config main.py:379; uploads.py:70/111/157/188. Every anchor is wrong; these comments are the repo's 'verified contract' convention, so wrong anchors actively mislead the next contract audit. Engine PR #390 (extract orchestration.py from main.py) is still OPEN (updated 2026-06-26) and will invalidate whatever numbers remain.",  
  "proposed_fix": "Replace hard line numbers with stable route anchors ? e.g. 'backend/main.py @app.post(\"/api/process\")' / 'backend/api/uploads.py @router.post(\"/api/upload/init\")' ? in one mechanical pass over client.ts + types.ts. Do it after (or regardless of) #390 so the anchors survive the god-module split.",  
  "effort": "S",  
  "tracked_in": "engine PR #390 (the split that worsens it)"  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training, product/strategy/licensing, real cloud spend, counsel ToS/DPA?

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code,

say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T03:02:11.398Z)

```
1      // Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the
single-origin
2      // appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ?
:8000).
3      // Every call carries a correlation id (header + logs) so a journey is traceable end to
end.
4      import { log, newCorrelationId } from "../lib/log";
5      import type {
6          RallySegment,
7          QueryResponse,
8          CompileResult,
9          EngineConfig,
10         EngineCapabilities,
11         ProcessResponse,
12         JobStatus,
13         UploadInitResponse,
14         UploadPartResponse,
15         UploadCompleteResponse,
16         VideoSummary,
17         VideosResponse,
18         AnnotationsResponse,
19         AssetResponse,
20     } from "../types";
21
22     /** Resolve the engine API base. Default same-origin `"/api"`. For a SPLIT deploy ? SPA
on
23     * Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel
(HOSTING_PLAN.md §2) ?
24     * set `VITE_API_BASE` at build time to the engine's API base **including the `/api`
path**
25     * (e.g. `https://api.khelsutra.guru/api`), and lock the engine's
`RALLY_CORS_ALLOW_ORIGINS` to the
26     * Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed
so
27     * `` `${base}/process` `` never double-slashes. */
28     export function apiBase(): string {
29         return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
30     }
31
32     const BASE = apiBase();
33
34     export class ApiError extends Error {
35         readonly cid?: string;
36         readonly status?: number;
37         readonly body?: string;
38         constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
39             super(message);
40             this.name = "ApiError";
41             this.cid = opts.cid;
42             this.status = opts.status;
43             this.body = opts.body;
44             if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
45         }
46     }
47
48     interface RequestOpts {
49         method?: string;
50         body?: string;
51         headers?: Record<string, string>;
52         /** Correlation id to thread through this call; auto-generated if absent. */
```

```

53     cid?: string;
54 }
55
56 async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
57     const cid = opts.cid ?? newCorrelationId();
58     const url = `${BASE}${path}`;
59     const method = opts.method ?? "GET";
60     log("info", "api.request", { cid, method, url });
61
62     let res: Response;
63     try {
64         res = await fetch(url, {
65             method,
66             body: opts.body,
67             headers: { "content-type": "application/json", "x-correlation-id": cid,
68                 ...(opts.headers ?? {}) },
69         });
69     } catch (err) {
70         // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
71         log("error", "api.network_error", { cid, method, url, error: String(err) });
72         throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
73     }
74
75     if (!res.ok) {
76         const body = await res.text().catch(() => "");
77         log("error", "api.http_error", { cid, method, url, status: res.status, body:
78 body.slice(0, 500) });
78         throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
79 res.status, body });
79     }
80
81     const data = (await res.json()) as T;
82     log("info", "api.response", { cid, method, url, status: res.status });
83     return data;
84 }
85
86 /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
87 for
87 * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
88 fails 400
88 * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
89 ApiError. */
89 export async function processVideo(
90     videoPath: string,
91     cid?: string,
92     locale?: string,
93     compute?: string,
94 ): Promise<ProcessResponse> {
95     // `locale` (the UI language) is recorded for PMF analytics (which markets use the
96 tool). `compute`
96 // is the compute-picker choice ("local" | "cloud") ? where DETECT runs; omitted ? the
97 server
97 // default. Both optional + omitted when absent (JSON.stringify drops undefined).
98 return request<ProcessResponse>("/process", {
99     method: "POST",
100     body: JSON.stringify({ video_path: videoPath, locale, compute }),
101     cid,
102 });
103 }
104
105 /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
106 export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
107     return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
108 });
108 }

```

```

109
110     /** List indexed videos, newest first ? the "my videos" library. GET /api/videos
[main.py:447].
111     * Reload-survival: a past match can be reopened without re-processing. Returns [] on
an empty box. */
112     export async function listVideos(cid?: string): Promise<VideoSummary[]> {
113         const data = await request<VideosResponse>("/videos", { method: "GET", cid });
114         return data.videos ?? [];
115     }
116
117     /** The source-video stream URL for annotation playback. GET /api/source/{video_id}
(P8). The engine
118     * serves a frame-identical proxy; labels stay keyed to the master's MD5
(STUDIO_MEDIA_LIFECYCLE D4). */
119     export function sourceUrl(videoId: string): string {
120         return `${BASE}/source/${encodeURIComponent(videoId)}`;
121     }
122
123     /** Persist finished rally labels as the annotator CSV. POST /api/annotations {video_id,
csv} (P8).
124     * Sent as the byte-compatible CSV (serializeRows) so the engine writes it verbatim
into the corpus. */
125     export async function postAnnotations(videoId: string, csv: string, cid?: string):
Promise<AnnotationsResponse> {
126         return request<AnnotationsResponse>("/annotations", {
127             method: "POST",
128             body: JSON.stringify({ video_id: videoId, csv }),
129             cid,
130         });
131     }
132
133     /** Store a video into the user's chosen medium + register it as a library asset (P4,
D14).
134     * POST /api/assets {uploaded_path? | source_uri?, medium_uri, sport?} ? 201 {video_id,
stored_uri}.
135     * Exactly ONE source: an `uploadedPath` (a just-uploaded local file ? a durable
MD5-keyed copy is
136     * put into the medium, the local upload kept as the working copy) OR a `sourceUri` (a
video already
137     * at rest, registered as-is). Feed `stored_uri` into processVideo(). Fast-fails 400
(bad medium /
138     * both-or-neither source), 403 (hosted posture), 404 (missing file), 507 (no space),
502 (put) ?
139     * surfaced as ApiError. */
140     export async function createAsset(
141         args: { uploadedPath?: string; sourceUri?: string; mediumUri: string; sport?: string
},
142         cid?: string,
143     ): Promise<AssetResponse> {
144         return request<AssetResponse>("/assets", {
145             method: "POST",
146             body: JSON.stringify({
147                 uploaded_path: args.uploadedPath,
148                 source_uri: args.sourceUri,
149                 medium_uri: args.mediumUri,
150                 sport: args.sport,
151             }),
152             cid,
153         });
154     }
155
156     /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
157     export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {

```

```

158     const data = await request<QueryResponse>("/query", {
159         method: "POST",
160         body: JSON.stringify({ video_id: videoId }),
161         cid,
162     });
163     return data.play_segments ?? [];
164 }
165
166 // Compile-job poll cadence (mirrors useIngestJob): a long ffmpeg stitch must not block
the request.
167 const COMPILE_POLL_MIN_MS = 1000;
168 const COMPILE_POLL_MAX_MS = 5000;
169 const COMPILE_POLL_BACKOFF = 1.5;
170 const COMPILE_POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' compile won't poll
forever
171
172 /** Stitch selected rallies into one reel. ASYNC (#329): POST /api/compile returns 202 +
a job_id;
173 * the ffmpeg stitch runs on the worker and we POLL GET /api/jobs/{id} for the reel ? a
long stitch
174 * would otherwise exceed the ~100 s edge/proxy timeout (524) while the reel actually
built. Resolves
175 * with the job's CompileResult on success; throws ApiError on a failed/timed-out
compile.
176 * `locale` (the UI language) localizes the reel watermark engine-side (omitted when
absent). */
177 export async function compileReel(
178     videoId: string,
179     segmentIds: number[],
180     outputFilename: string,
181     cid?: string,
182     locale?: string,
183 ): Promise<CompileResult> {
184     const c = cid ?? newCorrelationId();
185     const submitted = await request<ProcessResponse>("/compile", {
186         method: "POST",
187         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename, locale }),
188         cid: c,
189     });
190     return pollCompileJob(submitted.job_id, c);
191 }
192
193 /** Poll a submitted compile job to its terminal state. Returns the reel result on
success; throws an
194 * ApiError for an engine job-failure, a success-shaped FAILURE (done but the stitch
failed / no URL),
195 * or a poll timeout ? so the caller's existing classifyError/errorText mapping
surfaces it like ingest. */
196 async function pollCompileJob(jobId: string, cid: string): Promise<CompileResult> {
197     const startedAt = Date.now();
198     let delay = COMPILE_POLL_MIN_MS;
199     for (;;) {
200         const job = await getJob(jobId, cid);
201         if (job.status === "done") {
202             const result = (job.result ?? {}) as CompileResult;
203             if (result.status === "failed" || !result.download_url) {
204                 // A real stitch failure carries an engine message ? surface it as an `engine`
detail (status +
205                 // body set) so classifyError shows it verbatim, NOT the no-status "engine
unreachable" fallback.
206                 const msg = result.message || "Highlight stitching failed.";
207                 throw new ApiError(msg, { cid, status: 502, body: msg });
208             }
209             return result;

```

```

210     }
211     if (job.status === "failed") {
212         const msg = job.error || "Compile job failed.";
213         throw new ApiError(msg, { cid, status: 502, body: msg });
214     }
215     if (Date.now() - startedAt > COMPILE_POLL_CEILING_MS) {
216         throw new ApiError("Compile timed out.", { cid, status: 504, body: "Compile timed
out." });
217     }
218     await new Promise((resolve) => setTimeout(resolve, delay));
219     delay = Math.min(delay * COMPILE_POLL_BACKOFF, COMPILE_POLL_MAX_MS);
220 }
221 }
222
223 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
224 export function highlightUrl(videoId: string, filename: string): string {
225     return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
226 }
227
228 /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
229 export async function getConfig(cid?: string): Promise<EngineConfig> {
230     return request<EngineConfig>("/config", { method: "GET", cid });
231 }
232
233 /** What THIS box can do ? drives the first-run wizard. GET /api/capabilities
(PACKAGING_PLAN P2c). */
234 export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
235     return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
236 }
237
238 /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
239 export function minShotCount(cfg: EngineConfig): number | undefined {
240     return cfg.stitching?.min_shot_count;
241 }
242
243 // ?? chunked upload (backend/api/uploads.py) ?????????????????????????????????
244 // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
245 // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
246
247 /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[uploads.py:64]. */
248 export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
249     return request<UploadInitResponse>("/upload/init", {
250         method: "POST",
251         body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
252         cid,
253     });
254 }
255
256 /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[uploads.py:89]. */
257 export async function uploadPart(
258     uploadId: string,
259     index: number,
260     chunk: Blob,
261     cid?: string,
262     signal?: AbortSignal,
263 ): Promise<UploadPartResponse> {
264     const c = cid ?? newCorrelationId();

```

```

265     const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
266     log("info", "api.request", { cid: c, method: "PUT", url });
267     let res: Response;
268     try {
269         res = await fetch(url, {
270             method: "PUT",
271             body: chunk,
272             headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
273             signal,
274         });
275     } catch (err) {
276         log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
277     });
278         throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
279     }
280     if (!res.ok) {
281         const body = await res.text().catch(() => "");
282         log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
283     body: body.slice(0, 500) });
284         throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
285     body });
286     }
287     log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
288     return (await res.json()) as UploadPartResponse;
289 }
290
291 /** Finalize: returns {path, video_id} ? feed `path` to processVideo() [uploads.py:123].
292 */
293 export async function uploadComplete(uploadId: string, totalParts: number, cid?:
294 string): Promise<UploadCompleteResponse> {
295     return
296     request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
297         method: "POST",
298         body: JSON.stringify({ total_parts: totalParts }),
299         cid,
300     });
301 }
302
303 /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
304 [uploads.py:147]. */
305 export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
306     await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
307     cid });
308 }
309
310 export interface UploadProgress {
311     sentBytes: number;
312     totalBytes: number;
313 }
314
315 /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
316 complete.
317 * On ANY failure or abort it best-effort DELETES the server-side partial, then
318 rethrows so the
319 * caller can classify it (the engine's 400/413 detail survives). Returns {path,
320 video_id}.
321 * Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
322 recovery is a
323 * fresh init (re-pick the file), which the UI's retry does. */
324 export async function uploadFile(
325     file: File,
326     opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
327     = {},
328 ): Promise<UploadCompleteResponse> {
329     const cid = opts.cid ?? newCorrelationId();

```

```

317     const init = await uploadInit(file.name, file.size, cid);
318     // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
to NaN (which
319     // would silently upload zero parts) ? fall back to the engine's 95 MB default.
320     const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
init.max_part_mb : 95;
321     const partBytes = partMb * 1024 * 1024;
322     const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
323     log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
324     let sent = 0;
325     try {
326         for (let i = 0; i < totalParts; i++) {
327             if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
328             const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
329             await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
330             sent += chunk.size;
331             opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
332         }
333         const done = await uploadComplete(init.upload_id, totalParts, cid);
334         log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
335         return done;
336     } catch (err) {
337         log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
338         await uploadAbort(init.upload_id, cid).catch(() => {});
339         throw err;
340     }
341 }
342

```

3. user (2026-07-02T03:02:11.648Z)

```

1     // Types mirror the engine's verified contract (badminton-highlight-indexer).
2     // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3     // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5     export interface RallyMetadata {
6         /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7         shot_count?: number;
8         /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9         shot_count_confidence?: string;
10        detector?: string;
11        [key: string]: unknown;
12    }
13
14    export interface RallySegment {
15        /** The integer the UI selects and passes to /api/compile as segment_ids. */
16        id: number;
17        start_time: number;
18        end_time: number;
19        duration: number;
20        /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21        server: string | null;
22        /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23        receiver: string | null;
24        metadata: RallyMetadata;
25        /** motion-only fabricated sub-events ? DO NOT surface. */
26        events?: unknown[];
27    }
28
29    export interface QueryResponse {

```

```

30     play_segments: RallySegment[];
31 }
32
33 /** One row of the indexed-video library. GET /api/videos [backend/main.py:447] returns
the most
34 * recent first so the console survives a reload. Every field but `video_id` is
optional/defensive. */
35 export interface VideoSummary {
36     /** The MD5 join key (the contract field). Optional defensively ? an older/regressed
engine may
37     * send only the raw `id`; read it through `lib/video.ts::videoId`, never directly.
*/
38     video_id?: string;
39     /** Raw DB fields an engine may send instead of the enriched contract (resilience
fallbacks). */
40     id?: string;
41     filepath?: string | null;
42     match_name?: string | null;
43     local_path?: string | null;
44     fps?: number | null;
45     resolution?: string | null;
46     created_at?: string | null;
47 }
48
49 export interface VideosResponse {
50     videos: VideoSummary[];
51 }
52
53 /** POST /api/annotations ? persist finished rally labels for a video as the annotator
CSV (P8).
54 * The engine writes the CSV verbatim (byte-compatible with the pipeline), keyed by the
file MD5. */
55 export interface AnnotationsResponse {
56     status: string;
57     num_rows?: number;
58     csv_uri?: string;
59     message?: string;
60 }
61
62 export interface CompileResponse {
63     status: string;
64     /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
65     filepath: string;
66 }
67
68 /** The terminal RESULT of an async reel compile (#329): the job's `result` payload,
surfaced by
69 * GET /api/jobs/{id} once the stitch finishes. `status` is the pipeline verdict
("success" |
70 * "failed"); a "failed" result carries a `message`. The SPA builds the download URL
with
71 * highlightUrl() (split-origin-safe), but the engine also returns a same-origin
`download_url`. */
72 export interface CompileResult {
73     status: string;
74     filepath?: string;
75     download_url?: string;
76     message?: string;
77     video_id?: string;
78 }
79
80 export interface EngineConfig {
81     stitching?: {
82         /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
*/

```

```

83         min_shot_count?: number;
84         [key: string]: unknown;
85     };
86     [key: string]: unknown;
87 }
88
89 /** GET /api/capabilities [backend/main.py] ? an HONEST probe of what THIS box can do,
rendered by
90 * the first-run wizard (PACKAGING_PLAN.md P2c). Presence-booleans only (never the Gemini
key value).
91 * Every field optional/defensive: a missing field degrades the wizard gracefully. */
92 export interface EngineCapabilities {
93     api_version?: number;
94     engine_version?: string;
95     segmenters?: string[];
96     default_segmenter?: string;
97     ffmpeg?: { ffmpeg?: boolean; ffprobe?: boolean; ffmpeg_path?: string; ffprobe_path?:
string };
98     gpu?: { nvidia_smi_present?: boolean };
99     /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step.
*/
100     gemini_key_present?: boolean;
101     wasb_weights_present?: boolean;
102     /** `cloud_available` = can THIS server satisfy a per-request compute="cloud" dispatch
(a Cloud Run
103     * GPU Job)? The ingest compute-picker shows the "in the cloud" option only when
true. */
104     deployment?: { profile?: string; compute_target?: string; cloud_available?: boolean };
105     /** Which storage media THIS box can store INTO (P3). The medium picker renders only
`usable`
106     * backends; `free_gb` is `null` for a cloud bucket (no readable capacity). Absent on
older engines
107     * ? the SPA degrades to local-only (no picker). */
108     storage?: StorageCapabilities;
109     [key: string]: unknown;
110 }
111
112 /** One storable medium reported by GET /api/capabilities (P3). Presence/usable booleans
only ? no
113 * secrets. `id`/`kind` are the backend name (local/gcs/s3/gdrive). */
114 export interface StorageBackendInfo {
115     id: string;
116     kind: string;
117     label: string;
118     usable: boolean;
119     /** Free space in whole GB, or `null` when unknowable (a cloud bucket). */
120     free_gb?: number | null;
121 }
122
123 export interface StorageCapabilities {
124     default_medium?: string;
125     backends?: StorageBackendInfo[];
126 }
127
128 /** POST /api/assets ? 201 (P4). The video was stored into the chosen medium and
registered as a
129 * library asset keyed by the whole-file MD5 (`video_id`). `stored_uri` is the durable
location to
130 * feed into POST /api/process (a bucket/Drive URI, or a local path for "this
computer"). */
131 export interface AssetResponse {
132     video_id: string;
133     /** The medium backend the asset lives in (local | s3 | gcs | gdrive). */
134     medium: string;
135     /** The durable, MD5-keyed location ? pass as video_path to processVideo(). */

```

```

136     stored_uri: string;
137     /** True when a durable copy was made (uploaded ? remote medium); false when
registered in place. */
138     copied: boolean;
139     sport?: string | null;
140     status?: string;
141 }
142
143 /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
/api/jobs/{job_id}. */
144 export interface ProcessResponse {
145     job_id: string;
146     /** Always "queued" at submit. */
147     status: string;
148     /** Relative poll path, e.g. "/api/jobs/<id>". */
149     poll: string;
150 }
151
152 /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
sequential parts
153 * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
POST /api/process. */
154 export interface UploadInitResponse {
155     upload_id: string;
156     /** Per-part ceiling in MB ? slice the file into parts this big. */
157     max_part_mb: number;
158     next_part: string;
159 }
160 export interface UploadPartResponse {
161     upload_id: string;
162     received_parts: number;
163     received_bytes: number;
164 }
165 export interface UploadCompleteResponse {
166     /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
167     path: string;
168     video_id: string;
169     size_bytes: number;
170 }
171
172 /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
173 * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
(success|failed).
174 * A `done` job whose `result.status === "failed"` (or with no video_id) is a success-
shaped
175 * FAILURE ? the UI must not treat it as a load. */
176 export interface JobStatus {
177     job_id: string;
178     status: "queued" | "running" | "done" | "failed";
179     /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
percentage. */
180     progress?: string | null;
181     /** The canonical video_id, surfaced top-level on success (also in result.video_id).
*/
182     video_id?: string | null;
183     /** Execution error (worker crash / unexpected) when status === "failed". */
184     error?: string | null;
185     result?: { status?: string; video_id?: string; message?: string } | null;
186 }
187

```

4. user (2026-07-02T03:02:20.237Z)

```

258:@app.get("/", response_class=HTMLResponse)
379:@app.get("/api/config")

```

```

391:@app.get("/api/health")
419:@app.get("/api/version")
429:@app.get("/api/capabilities")
521:@app.get("/api/videos")
537:@app.get("/api/reels")
570:@app.get("/api/reels/{filename}")
623:@app.get("/api/source/{video_id}")
663:@app.post("/api/annotations")
727:@app.post("/api/assets", status_code=201)
1355:@app.post("/api/process", status_code=202)
1431:@app.get("/api/jobs/{job_id}/cost")
1446:@app.get("/api/videos/{video_id}/cost")
1457:@app.get("/api/jobs/{job_id}")
1492:@app.get("/api/highlights/{video_id}/{filename}")
1523:@app.post("/api/query")
1594:@app.post("/api/compile", status_code=202)

```

5. user (2026-07-02T03:02:21.322Z)

```

70:@router.post("/api/upload/init")
111:@router.put("/api/upload/{upload_id}/part/{index}")
157:@router.post("/api/upload/{upload_id}/complete")
188:@router.delete("/api/upload/{upload_id}")

```

6. user (2026-07-02T03:02:43.753Z)

CORS for the web UI. `allow_origins='*' with allow_credentials=True is rejected by browser fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so credentials off. The origin list is configurable via the env var above (default ["*"]) so a hosted deployment lock it to its origin; the middleware is fixed at startup.`

```

app.add_middleware(
    CORSMiddleware,
    app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
)

```

```

@app.get("/", response_class=HTMLResponse)
def serve_index():
    index_path = os.path.join(_UI_DIR, "index.html")
    if os.path.exists(index_path):
        with open(index_path, "r", encoding="utf-8") as f:
            _run_claimed_job,
            _schedule_next_drain_if_waiting,
    )

```

Legacy thin wrapper for any remaining direct calls during transition.

New code should import `load_app_config` / `AppConfig` from `backend.config` directly.

```

def load_config(config_path: str = "config.json") -> Dict[str, Any]:
    cfg = load_app_config(config_path)
    return cfg.model_dump(mode="json")

class CompileRequest(BaseModel):
    video_id: str
    segment_ids: List[int]
    output_filename: str
    # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel watermark
    # (text + script font). Optional ? absent/unknown keeps the configured default (en
    behavior).
    locale: Optional[str] = None

```

```
def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
```

7. user (2026-07-02T03:02:45.225Z)

```
        self._backup_before_migrate(conn, version)

    if version < 1:
        self._migrate_to_v1(cursor)

    if version < 2:
        self._migrate_to_v2(cursor)

    if version < 3:
        self._migrate_to_v3(cursor)

    if version < 4:
        self._migrate_to_v4(cursor)

    if version < 5:
        self._migrate_to_v5(cursor)

    if version < 6:
        self._migrate_to_v6(cursor)

    if version < 7:
96:         """Initializes tables for videos, play_segments, and events."""
190:         """Create the videos / play_segments / events base tables (idempotent)."""
204:         CREATE TABLE IF NOT EXISTS play_segments (
226:             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
251:             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON DELETE
SET NULL
```

8. user (2026-07-02T03:02:46.643Z)

```
f9eea14 web: resilient video-summary readers + show fps in the library (#140)
8667dbf P5: SPA storage-medium picker (store a video into your medium) (#137)
cc06c32 feat(web): P8 annotate-in-UI ? AnnotatePanel over the vendored core (flagged) (#128)
39c1349 feat(web): P6 ? "Your videos" library (reopen a stored video) (#124)
ed33a51 feat(build): poll the async /api/compile job for the reel (#329) (#108)
f67dfe7 feat(ingest): compute-picker ? choose local vs cloud GPU per run (item 3b) (#104)
7ae9cee feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages)
(#88)
c1848df feat(web): LW2 ? send the UI locale to /api/compile (watermark) + /api/process
(analytics) (#73)
ca619fb feat(web): T4 ? in-UI file upload (the runnable-tool keystone) (#66)
8bbc557 feat(web): VITE_API_BASE ? let the SPA target a split api.* origin (default same-origin
/api) (#64)
4525c5d feat(web): B-P2 ? in-UI bucket ingest (point at a gs:// video ? process ? load) (#56)
aac7e0c feat(web): per-rally MVP P1 ? SPA scaffold + typed engine API client (#14)
f9eea14 web: resilient video-summary readers + show fps in the library (#140)
```

master...origin/master

9. user (2026-07-02T03:03:02.940Z)

master...origin/master

```
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
```

10. user (2026-07-02T03:03:03.953Z)

```
{"files":["backend/main.py","backend/orchestration.py","mypy.ini"],"state":"OPEN","title":"refac
tor(D1/P10): extract orchestration.py from main.py (god-module split, step
```

```
1)","updatedAt":"2026-06-26T14:37:18Z"}
```

11. user (2026-07-02T03:04:06.535Z)

Structured output provided successfully